Direct Memory Access

Chapter Contents

Overview

This chapter presents the ***direct memory access (DMA)*** controller, a peripheral that is able to take control of the MCU's address and data bus in order to transfer data directly with read and write hardware operations, rather than relying on explicit load and store instructions in a program. Peripheral events (e.g. timer overflows) that can trigger interrupt requests can also be used to trigger DMA transfers. The DMA controller can eliminate simple ISRs, reducing the amount of software that the MCU must execute, which improves performance and responsiveness. In this chapter we see how to use DMA to copy memory data quickly, and also how to generate an analog waveform from data stored in memory using the DAC and a timer.

> **direct memory access (DMA)**
> *Type of memory access performed by peripheral hardware without using program instructions*

Concepts

A basic DMA transfer occurs as follows:

- A trigger event starts the transfer. This may be an explicit software write to a start field in a control register of the DMA controller, or an event such as a timer overflow or an ADC conversion completion.
- The DMA controller takes control of the MCU's address and data buses and control lines in order to read a data item from the source location (which is specified in a source address register). This source may be memory or a memory-mapped peripheral.
- The DMA controller then uses the address and data buses and control lines in order to write the data to the destination location (which is specified in a destination address register). The DMA controller releases the buses and control lines for the CPU to use.
- The DMA controller increments the source and destination address registers so the next item is addressed for the next transfer.
- The DMA controller updates a count register that tracks the number of items transferred. If there are more items to transfer, this process repeats.
- The DMA controller will indicate the final transfer has completed by setting a status flag and triggering an interrupt (if enabled).

There are two important variations of this basic transfer process:

- The address registers contain the base address of the source and destination. These address registers are usually incremented (by the data size) after each transfer. However, in some cases we wish to have the DMA controller to copy the same value into every destination, or read successive values from the same source. To enable this, each address register can be configured to advance or remain unchanged.
- Some DMA controllers offer two transfer modes. In the round-robin (cycle-stealing or time-sharing) mode, the DMA controller shares the bus with other masters (e.g. the CPU core), taking control to transfer one item, and then yielding the bus. In the continuous (burst) mode, the DMA controller takes over the bus to transfer all data items non-stop until the transfer is complete. The first mode provides latency fairness among DMA masters at the price of limiting peak bandwidth (throughput).

One obvious use for DMA is to transfer or fill a block of memory quickly. For example, this could be useful for erasing a buffer, initializing a data structure, or copying an image into a frame buffer. However, there are more sophisticated uses possible when combined with other peripherals. For example, DMA can be used to transfer a data value from a waveform buffer to the DAC in order to generate an analog waveform. Each transfer is controlled by a timer overflow, allowing precise timing with minimal CPU overhead. We will implement this example later in this chapter.

Using DMA transfers can also help reduce the energy or power used by the system. Because DMA transfers reduce the processing required of the CPU, it may be possible to place the CPU in a low-power sleep mode. Required peripherals will remain active, whereas the rest of the MCU will be disabled or powered down, reducing both the power and energy consumption of the system. Another option is to keep the CPU active but running at a lower clock frequency, which reduces power consumption.

STM32F091RC DMA Controller and Routing Peripherals

As shown in Figure 9.1, the STM32F091RC MCU has two DMA controllers (DMA1 and DMA2) which are connected to a bus matrix. This bus matrix allows simultaneous operation of multiple masters (Cortex-M0 core and DMA controllers) when they are accessing different slaves (e.g. memories, AHB devices). When multiple masters try to access the same slave, a round-robin hardware arbiter lets a master transfer only one data item at a time before advancing to the next master. This fairness ensures the CPU will get to execute code often, although peak DMA throughput may be limited. A DMA application note explains concepts, provides design guidance and analyses performance [1], while the controller implementation details are described in the reference manual [2].

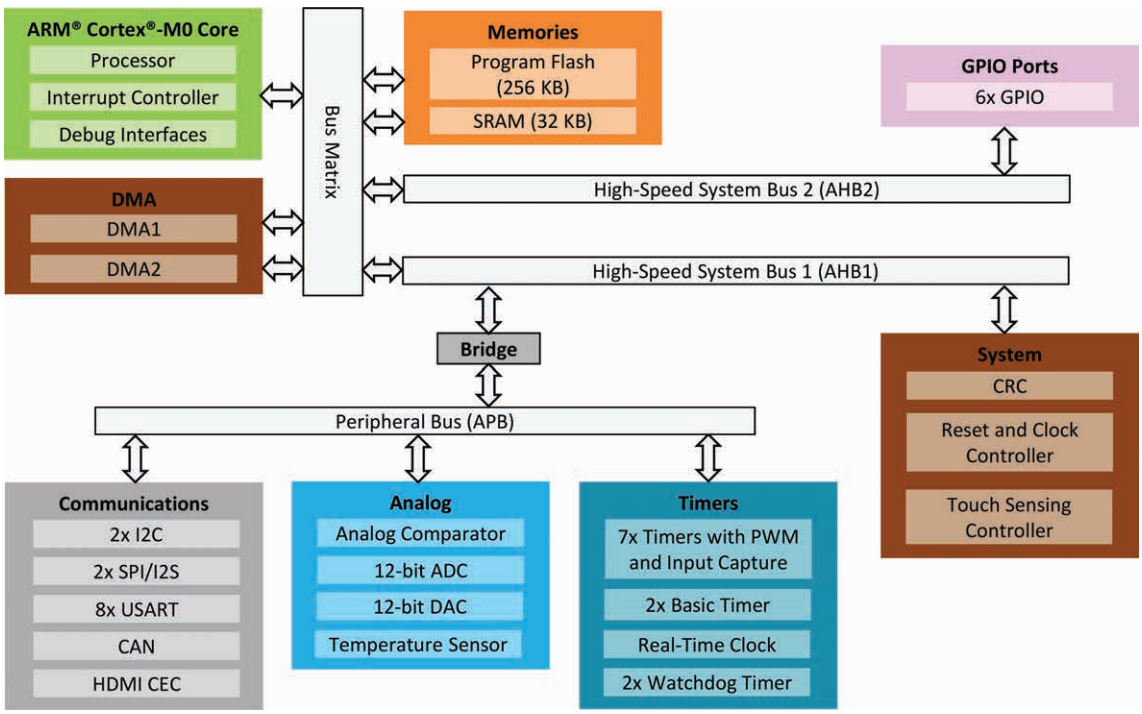

Figure 9.1 DMA controllers (left side) can use bus matrix to transfer data between devices. Only masters (CPU core and DMA controllers) can control the bus.

The two DMA controllers have multiple channels. DMA1 has seven channels, numbered 1 through 7. DMA2 has five channels, numbered 1 through 5. Figure 9.2 shows the structure of the channel y of a DMA controller channel.

The address registers are called CMARy (memory address register) and CPARy (peripheral address register). Despite these names, all four types of transfer are possible: memory to peripheral, peripheral to memory, between memory, or between peripherals. The control bit DIR determines the transfer direction. For example, if DIR is 0, CPARy specifies the source and CMARy the destination.

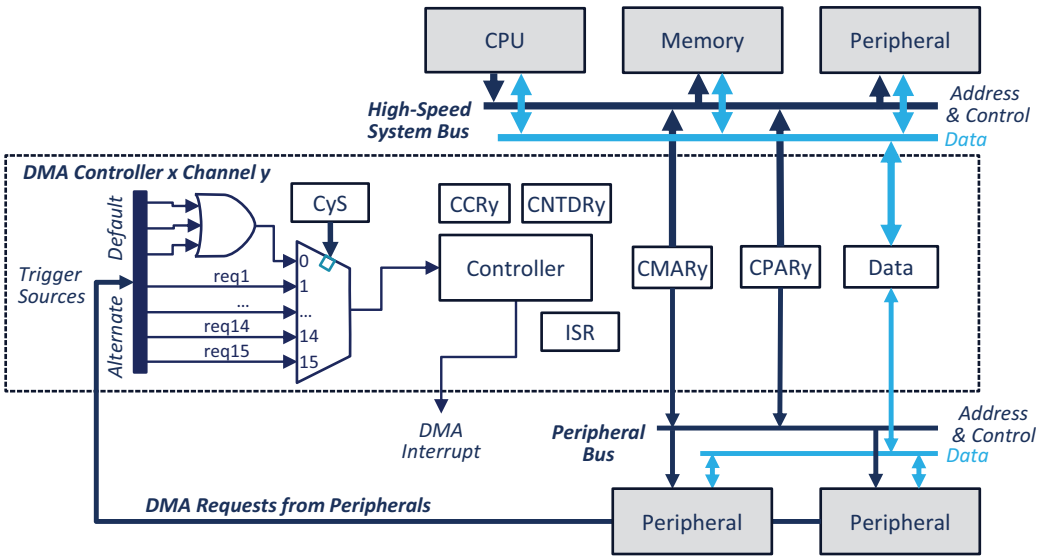

Figure 9.2 Events from other peripherals can trigger a DMA controller channel.

The channel y number of data register (CNDTRy) holds the number of data items remaining to be read from the source and transferred. A data item is defined to be the same size as the source data.

DMA transfers with peripherals can be started by a hardware trigger event, but memory-to-memory transfers must be started by a software write operation. When the hardware trigger is used, the channel's trigger routing multiplexer selects one of many possible trigger event requests from the other peripherals (based on the control field CyS) and provides it to the DMA controller for that channel.

When the trigger occurs, the controller first reads the data specified by the source address register (CPARy or CMARy, depending on DIR). The source and destination can have the same or different data sizes, with options of one, two or four bytes. That data may need to be buffered and reformatted if source and destination are of different sizes, or if either is unaligned. The controller then writes the data to the location indicated by the destination address register for channel y (CPARy or CMARy).

The controller then decrements the CNDTRy data transfer counter. If it is not zero, the controller will repeat this sequence. If it is zero, then all transfers have been completed, so a status flag (TCIFy) is set and an interrupt can be generated. There are additional features available such as half-transfer complete, circular transfer and error detection. They are described below and in the reference manual.

DMA Request Routing and Trigger Sources

The channel routing multiplexer selects the hardware source to trigger the DMA channel. The DMA Channel Selection Register DMAx_CSELR (Figure 9.3) controls each channel's input multiplexer.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	C7S				C6S				C5S			
Reset					0	0	0	0	0	0	0	0	0	0	0	0
Access					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	C4S				C3S				C2S				C1S			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 9.3 DMAx_CSELR control register selects trigger source for DMA channel.

Each 4-bit CyS field selects the trigger source of the channel y. Table 9.1 shows the default trigger source selection for channel 1 and 2 of DMA1, when the fields are all cleared to 0000 (as after reset). Note that some channels have multiple peripherals listed. These are logically ORed together: if any of the listed peripheral requests DMA service, the DMA will be triggered. So when configuring the peripheral, be sure to enable DMA request generation for only one peripheral per channel.

Table 9.1 Default trigger sources for DMA1 (when CyS = 0000). Note that DMA2 has no default trigger sources and requires selecting a source with DMA2_CyS.

Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
TIM2_CH3	TIM2_UP TIM3_CH3	TIM3_CH4 TIM3_UP	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	–	–
–				TIM2_CH1	–	–
–	–	–		TIM15_CH1 TIM15_UP TIM15_TRIG TIM15_COM	–	–
ADC	–	TIM6_UP DAE_ Channel1	TIM7_UP DAC_Channel2		–	–
–	USART1_TX	USART1_RX	USART2_TX	USART2_RX	USART3_RX	USART3_TX
	–	–	–	–	USART4_RX	USART4_TX
–	SPI1_RX	SPI1_TX	SPI2_RX –	SPI2_TX –	–	–
–	12C1_TX	12C1_RX	I2C2_TX –	I2C2_RX –	–	–
–	TIM1_CH1	TIM1_CH2	–	TIM1_CH3 –	–	–
–	–	TIM2_CH2	TIM2_CH4	–	–	–
TIM17_CH1 TIM17_UP	_	TIM16_CH1 TIM16_UP	TIM3_CH1 TIM3_TRIG	_	_	_

Changing CyS from 0000 to a different value will allow selection of different trigger sources for a given channel. For further details, please see the section **DMA1/DMA2 controllers on STM32F09x devices** in the reference manual.

DMA Channels

Each DMA channel y has several control registers, as shown in Figure 9.2. Each channel has a DMAx_CCRy register, shown in Figure 9.4, which configures the operation of the channel.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	MEM2MEM	PL		MSIZE		PSIZE		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN
Reset		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 9.4 DMAx_CCRy register defines DMA operation.

Let us examine the fields that define the basic aspects of the transfer.

- MSIZE and PSIZE specify the data sizes of the memory and the peripheral: 8 bits (00), 16 bits (01), or 32 bits (10). The memory and peripheral sizes do not need to match. If the source data is narrower than the destination data, the source data is zero-padded to fill the destination. If the source data is wider than the destination data, only the least-significant byte or halfword is written, with the remaining bytes discarded. More details appear in the MCU reference manual [2].
- MINC and PINC, when set to one, cause the memory address register or peripheral address register to increment by the corresponding data size (one, two or four bytes) after each transfer. A value of zero will result in no incrementing of that address register.
- The DIR field selects the direction of the transfer. If DIR is zero, data is transferred from the location pointed to by CPARx to that pointed to by CMARx (peripheral to the memory). If DIR is one, data is transferred in the opposite direction (CMARx to CPARx).
- If MEM2MEM is set, a memory to memory transfer is selected; no peripheral trigger is needed. Memory to memory mode may not be used at the same time as circular mode.
- CIRC enables (1) or disables (0) the circular mode. In circular mode, DMAx_CNTDRy is automatically reloaded when it reaches zero and the DMA transfers can continue. The internal registers in the MCU are also reloaded with the base address from the DMAx_CNDTRy or DMAx_CMARy registers. This allows us to use a circular buffer to handle continuous data flows.
- PL sets the priority level of the DMA channel. If several channels are triggered at the same time, the channel with the highest PL value is executed first. If two channels have the same PL value, the channel with the lowest number is executed first (i.e. if channel 1 and channel 2 have the same priority level, channel 1 is executed first).
- Writing a one to EN enables the channel. If MEM2MEM is 1 (selecting memory to memory mode), then enabling the channel starts the DMA transfer without awaiting a peripheral trigger.

There are three types of interrupt possible based on the interrupt enable fields:

- Error: If TEIE is set, an interrupt is raised when an error has occurred during a transfer.
- Half Transfer: If HTIE is set, an interrupt is raised after half the data has been transferred.
- Transfer Complete: If TCIE is set, an interrupt is raised when the transfer is completed.

Each DMA controller has one DMAx_ISR register, shown in Figure 9.5, which holds status flags for all the channels.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	TEIF7	HTIF7	TCIF7	GIF6	TEIF6	HTIF6	TCIF6	GIF6	TEIF5	HTIF5	TCIF5	GIF5
Reset					0	0	0	0	0	0	0	0	0	0	0	0
Access					r	r	r	r	r	r	r	r	r	r	r	r

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	TEIF4	HTIF4	TCIF4	GIF4	TEIF3	HTIF3	TCIF3	GIF3	TEIF2	HTIF2	TCIF2	GIF2	TEIF1	HTIF1	TCIF1	GIF1
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Figure 9.5 DMAx_ISR register holds status flags for all the channels.

- If an error has occurred during the transfer for the channel y, TEIFy is set.
- When half the data has been transferred for the channel y, HTIFy is set.
- When the transfer is completed for the channel y, TCIFy is set.
- GIFy is set if any event has occurred for the channel y (transfer error, half-transfer or transfer completed).

To clear the event flag, a one should be written in the Interrupt Flag Clear Register (DMAx_IFCR) in the event corresponding field.

Basic DMA Configuration and Use

The following steps are used to configure and use the DMA controller:

- Enable clock gating to the DMA module by setting DMAEN (DMA1EN) or DMA2EN in RCC_AHBENR register.
- Load DMAx_CPARy and DMAx_CMARy register with the peripheral and memory address.
- Load DMAx_CNDTRy with the number of data to transfer.
- Configure the channel priority level in DMAx_CCRy PL field.
- Configure the direction, the circular buffer, the memory to memory mode and the peripheral and memory data size in DMAx_CCRy.
- If the hardware trigger is used, configure the channel trigger routing in DMAx_CSELR.

- The interrupt can be enabled in the DMAx_CCRy to generate an interrupt at the end of the transfer or if an error has occured or the transfer has been completed by setting TEIE, HTIE or TCIE.
- Enable the DMA channel by setting the EN field in the DMAx_CCRy register. If the memory to memory mode has been chosen, setting EN to one starts the transfer. Otherwise, the transfer will start when the chosen hardware event triggers the DMA request.

Now the system must await the end of the transfer. For polling, wait until the TCIFy flag in DMAx_ISR changes to one. For interrupts, one of the DMA handlers will run after the transfer completes depending on the DMA and channel chosen.

Examples

Let's examine two different ways to use the DMA controller: copying data and generating an analog waveform. Figure 9.6 shows the signal locations of these examples on the Nucleo-64 board.

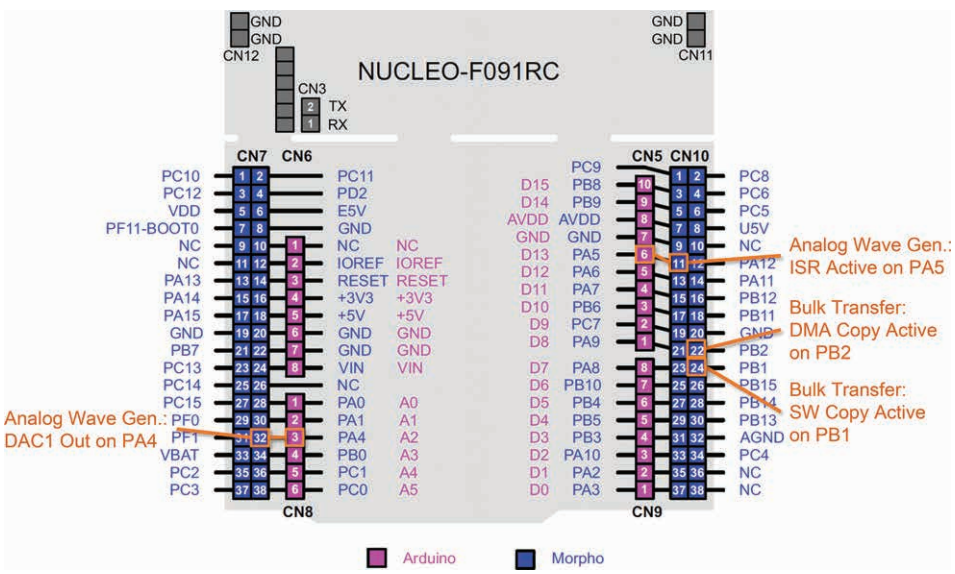

Figure 9.6 Signal locations for example code (Bulk Data Transfer and Analog Waveform Generator).

Bulk Data Transfer

How quickly can the processor copy a block of data in memory? Let us start with a simple software solution. Listing 9.1 shows the C source code to copy `ARR_SIZE` words from source array s to destination array d in memory.

```
void Test_SW_Copy(void) {
  uint32_t i;

  GPIOB->BSRR = GPIO_BSRR_BS_1;
  for (i = 0; i < ARR_SIZE; i++) {
    d[i] = s[i];
  }
  GPIOB->BSRR = GPIO_BSRR_BR_1;
}
```

Listing 9.1 C code to copy data from source array `s` to destination array `d`.

Listing 9.2 shows additional code which supports Listing 9.1 by initializing the arrays and confirming correct copying.

```
#define ARR_SIZE (256)
uint32_t s[ARR_SIZE], d[ARR_SIZE];

void Init_Arrays(void) {
  uint32_t i;
  for (i = 0; i < ARR_SIZE; i++) {
    s[i] = i;
    d[i] = 0;
  }
}

void Compare_Arrays(void) {
  uint32_t i;
  for (i = 0; i < ARR_SIZE; i++) {
    if (d[i] != s[i]) {
      while (1);        // stop here on error
    }
  }
}
```

Listing 9.2 Support code to define, initialize and compare arrays.

```
0x08000798 2000     MOVS     r0,#0x00
0x0800079A 0081     LSLS     r1,r0,#2
0x0800079C 585A     LDR      r2,[r3,r1]
0x0800079E 1C40     ADDS     r0,r0,#1
0x080007A0 506A     STR      r2,[r5,r1]
0x080007A2 28FF     CMP      r0,#0xFF
0x080007A4 D9F9     BLS      0x0800079A
```

Listing 9.3 Portion of assembly code generated by compiler for code to copy data.

Listing 9.3 shows part of the assembly code which the compiler generated for Listing 9.1. Register R0 counts the number of words to copy. Register R1 holds the byte offset of current word, which is 4 * the current word number. The `LDR` instruction loads the data word from source memory starting at byte address R3+R1. `ADDS` adds one to the current word number. `STR` stores that data word to destination memory starting at byte address R5+R1. `CMP` and `BLS` repeat the loop until all 256 words have been copied. The loop consists of six instructions (`LSLS`, `LDR`, `ADDS`, `STR`, `CMP` and `BLS`). Let us assume each instruction will take one clock cycle to execute, so the loop should take six clock cycles per iteration. The time needed for 256 iterations of six clock cycles each at 48 MHz is 32 µs.

We run the code on the MCU and measure the timing, finding that it takes 80 µs to copy 256 words. This translates to a transfer rate of 3.2 million words/second, or 15 clock cycles per loop iteration.

Why does the loop take fifteen cycles instead of six? The CPU's technical reference manual details the number of cycles needed to execute each type of instruction. Most instructions take only one cycle, but some take more. `LD` and `ST` each take two cycles (and would take more if they had more registers to load or store). `BLS` take three cycles each time the branch is taken back to `|L1.6|`, but only one cycle when it is not taken. Note that the loop can be accelerated through compiler optimization, which is outside the scope of this text.

Let's use the DMA controller to get rid of this loop and its instruction overhead. We will use channel 1 of the DMA1 controller to perform bulk transfers of words using memory to memory transfer and polling for completion detection.

The function `Test_DMA_Copy` in Listing 9.4 sets a debug bit, enables the clock for the DMA module, and then configures DMA_CCR1 to increment both the source and destination pointers and to transfer 32 bits at a time. The function then configures the DMA controller for the specific transfer. In memory to memory mode, the code stores the source and destination pointers in the peripheral and memory address registers. The code stores the data count in the count register. Next the code starts the transfer and polls the TCIF flag until it is set by the DMA controller, indicating the transfer has completed, and then clears the transfer complete flag and debug bit.

```
void Test_DMA_Copy(void) {
  GPIOB->BSRR = GPIO_BSRR_BS_2;
  RCC->AHBENR |= RCC_AHBENR_DMA1EN;

  DMA1_Channel1->CCR = DMA_CCR_MEM2MEM | DMA_CCR_MINC |
    DMA_CCR_PINC | DMA_CCR_DIR;
  MODIFY_FIELD(DMA1_Channel1->CCR, DMA_CCR_MSIZE, 2);
  MODIFY_FIELD(DMA1_Channel1->CCR, DMA_CCR_PSIZE, 2);
  MODIFY_FIELD(DMA1_Channel1->CCR, DMA_CCR_PL, 3);

  DMA1_Channel1->CNDTR = ARR_SIZE;
  DMA1_Channel1->CMAR = (uint32_t) s;
  DMA1_Channel1->CPAR = (uint32_t) d;

  DMA1_Channel1->CCR |= DMA_CCR_EN; // Enable DMA transfer
  while (!(DMA1->ISR & DMA_ISR_TCIF1))
    ;
  DMA1->IFCR = DMA_IFCR_CTCIF1; // Clear transfer complete flag
  GPIOB->BSRR = GPIO_BSRR_BR_2;
}
```

Listing 9.4 Code to use DMA controller to copy 32-bit words quickly.

The code in Listing 9.5 copies data between the arrays repeatedly, first using software and then DMA. You can monitor the time for each transfer on debug bits PB1 (software copy) and PB2 (DMA copy).

Running this code and measuring the timing of the debug bit using an oscilloscope reveals that it takes about 26.8 µs for the DMA controller to transfer 256 words. This means the transfer rate is 9.5 million words per second (38 megabytes/second). The core, DMA controller and AHB bus are all running at 48 MHz; why isn't the transfer rate faster?

The application note explains the delays involved and the resulting timing performance [1]. Multiple steps are needed to perform a DMA transfer: arbitration, address computation, SRAM read, SRAM write, and an acknowledgement handshake.

Arbitration further reduces throughput in this code: both the CPU core and the DMA try to access memory, so the arbiter gives permission alternately to the CPU core and the DMA controller. Although the CPU is just executing a busy-wait loop awaiting the transfer completion flag to be set, it still needs to fetch instructions and access the DMA register IFCR to do this.

```
int main(void) {
  Set_Clocks_To_48MHz();
  Init_GPIO();
  while (1) {
    Init_Arrays();
    Test_SW_Copy();
    Compare_Arrays(); // Stop if error

    Init_Arrays();
    Test_DMA_Copy();
    Compare_Arrays(); // Stop if error
  }
}
```

Listing 9.5 `main` function used to compare copy speeds.

Compare this with the software solution, which manages only 5.3 million words per second due to the overhead of executing instructions (to calculate byte offset, load and store values, increment the counter, and conditionally branch to repeat the loop).

Analog Waveform Generation

The second example targets the running example of the analog waveform generator introduced in Chapter 6 that uses the digital-to-analog converter (DAC). In Chapter 7 we saw how to use the timer to generate a regular interrupt. The timer ISR updates the DAC in order to generate the waveform. Using an ISR provides precise timing while separating the waveform generator code from the rest of the program.

In this chapter we will remove the timer ISR and use channel 3 of the DMA2 controller to transfer a data item from a memory buffer to the DAC every 10 µs. Note that the DMA controller can copy data but cannot generate it. We therefore will precompute the waveform samples and store them in an array from which the DMA controller will read.

Using DMA will reduce CPU loading by eliminating the timer ISR. Using the DMA to transfer data will also improve the timing stability even further, as the transfers will not be delayed by other ISRs (which might happen with the timer ISR approach).

Design

Figure 9.7 shows the sequence of events in waveform generation. The timer peripheral in the first column (TIM) generates a periodic event to trigger the transfer, rather than an ISR that was used in Chapter 7. The DAC will generate a DMA request. The DMA controller is configured

to transfer one sample (16 bits) per request. Circular mode is enabled, so the DMA automatically reloads the memory address and the number of data to transmit, making the next trigger repeat the DMA transfer process. The DMA controller will generate an interrupt after performing the last transfer to the DAC. This is not necessary for circular mode, but allows us to add a debug signal to confirm system operation.

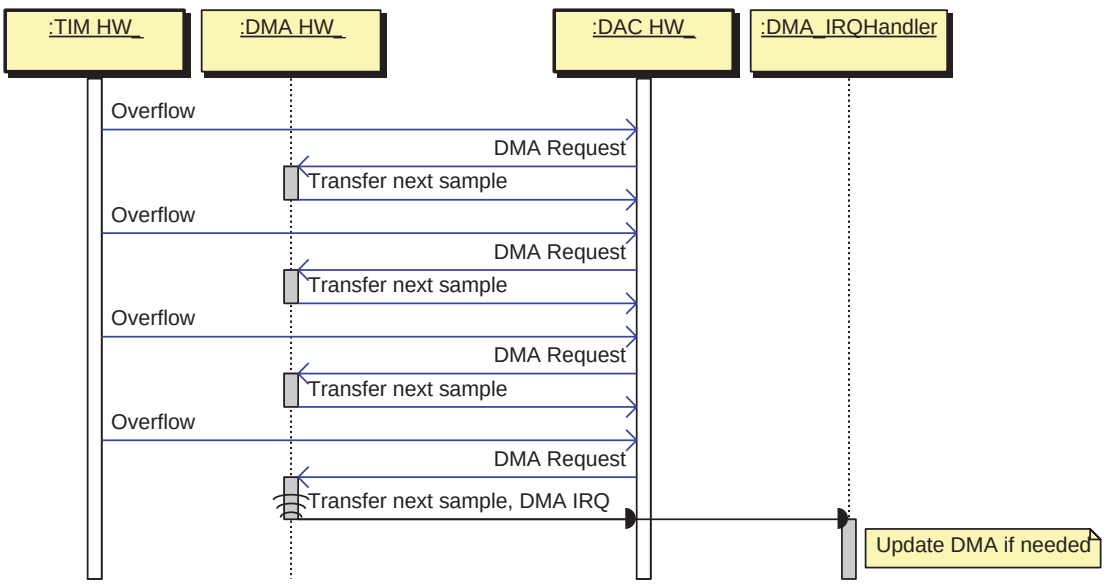

Figure 9.7 Sequence of events that generate analog waveform using timer, DMA, and DAC. The diagram shows DMA IRQ occurring after 4 transfers for readability, but in code IRQ occurs after 512 transfers.

The top-level code (in Listing 9.6) initializes the LEDs, the DAC, the sample array (`TriangleTable`), the DMA system, and the timer TIM. It then starts the DMA system. At this point all waveform generation work is performed by the DMA system and its ISR, so the function can return, do other work, or even enter an infinite loop (as shown here).

```
int main(void) {
  Set_Clocks_To_48MHz();
  Init_GPIO_RGB();
  Control_RGB_LEDs(0, 0, 0);
  Init_TriangleTable();
  Init_DMA_For_Playback();
  Init_TIM();
  Init_DAC();
  Start_DMA((uint32_t *) TriangleTable, NUM_STEPS);
  while (1); // PA5 low indicates ISR activity
}
```

Listing 9.6 Top-level code for generating analog waveform.

Listing 9.7 shows the data sample array called `TriangleTable`, and its initialization function `Init_TriangleTable`. This function takes advantage of the symmetry of the triangle wave to load up the array from both the front and back in each loop iteration.

```
#define MAX_DAC_CODE (4095)
#define NUM_STEPS (512)

uint16_t TriangleTable[NUM_STEPS];
void Init_TriangleTable(void) {
  unsigned n, sample = 0;

  for (n = 0; n < NUM_STEPS / 2; n++) {
    sample =
      (n * (MAX_DAC_CODE + 1) /
            (NUM_STEPS / 2));
    // Fill in from front
    TriangleTable[n] = sample;
    // Fill in from back
    TriangleTable[NUM_STEPS - 1 - n] = sample;
  }
}
```

Listing 9.7 Code to initialize data buffer `TriangleTable` with waveform samples.

```
#define F_TIM_CLOCK (48UL*1000UL*1000UL) // 48 MHz
#define F_TIM_OVERFLOW (100UL*1000UL) // 100 kHz
#define TIM_PRESCALER (16)
#define TIM_PERIOD (F_TIM_CLOCK/(TIM_PRESCALER*F_TIM_OVERFLOW))

void Init_TIM(void) {
  RCC->APB1ENR |= RCC_APB1ENR_TIM6EN;
  // Set up prescaler and counter to get 100 kHz
  TIM6->ARR = TIM_PERIOD-1;
  TIM6->PSC = TIM_PRESCALER-1;
  // Enable DMA request on update
  TIM6->DIER = TIM_DIER_UDE;
  // Enable counting
  TIM6->CR1 |= TIM_CR1_CEN;
}
```

Listing 9.8 Code to initialize timer TIM6 to generate DMA request every 10 µs (100 kHz).

Listing 9.8 shows the code to initialize TIM6 to request a DMA transfer every 10 µs based on a 48 MHz clock input to the timer and a prescaler setting of 16.

```
void Init_DMA_For_Playback(void) {
  // Enable DMA2 clock
  RCC->AHBENR |= RCC_AHBENR_DMA2EN;
  // Memory to peripheral mode, 16-bit data
  DMA2_Channel3->CCR = DMA_CCR_MINC | DMA_CCR_DIR | DMA_CCR_CIRC;
  MODIFY_FIELD(DMA2_Channel3->CCR, DMA_CCR_MSIZE, 1);
  MODIFY_FIELD(DMA2_Channel3->CCR, DMA_CCR_PSIZE, 1);
  MODIFY_FIELD(DMA2_Channel3->CCR, DMA_CCR_PL, 3);
```

```
  NVIC_SetPriority(DMA1_Ch4_7_DMA2_Ch3_5_IRQn, 3);
  NVIC_ClearPendingIRQ(DMA1_Ch4_7_DMA2_Ch3_5_IRQn);
  NVIC _ EnableIRQ(DMA1_Ch4_7_DMA2_Ch3_5_IRQn);
  // DMA2 Channel 3 requests come from TIM6_UP update (CxS = 0001)
  MODIFY_FIELD(DMA2->CSELR, DMA_CSELR_C3S, 1);
}
```

Listing 9.9 Code to initialize DMA system for playing back waveform.

Listing 9.9 shows the code to initialize the DMA controller for waveform playback. The function enables the clock gating for the DMA2 module. It configures the channel to transfer 16-bit words from memory to peripheral, increment the memory address but not the peripheral address, and perform one transfer when a peripheral request is received. The circular mode is chosen such that at the end of the transfer, the data count and the memory address are automatically reloaded by the hardware. The NVIC is configured to accept DMA interrupt requests, and finally the peripheral request trigger for DMA2 channel 3 is connected to the Timer 6 update signal.

```
void Start_DMA(uint32_t * source, uint32_t length) {
  DMA2_Channel3->CCR |= DMA_CCR_TCIE;
  DMA2_Channel3->CNDTR = length;
  DMA2_Channel3->CMAR = (uint32_t) source;
  // Peripheral: address of DAC->DHR12R1 data register
  DMA2_Channel3->CPAR = (uint32_t) &(DAC->DHR12R1);
  DMA2_Channel3->CCR |= DMA_CCR_EN;
}
```

Listing 9.10 Code to start or restart DMA playback of specific data buffer to DAC.

Listing 9.10 shows the code to start the waveform playback using DMA. The function is passed a pointer to the beginning of the source data (`TriangleTable` in this example) and the number of samples to transfer. The data count register is loaded with the number of data to transfer. The memory address register is loaded with the address of the data buffer, and the destination address is loaded with the address of the DAC1 12-bit right-aligned data register. Finally the function enables the DMA channel.

```
void DMA1_Ch4_7_DMA2_Ch3_5_IRQHandler(void) {
  // Red LED on
  Control_RGB_LEDs(1, 0, 0);
  if (DMA2->ISR & DMA_FLAG_TC3) {
    // DMA2 Channel 3 transfer complete, so do something
    // if needed here. Restart, etc.
  }
  DMA2->IFCR |= DMA_FLAG_TC3 | DMA_FLAG_HT3 | DMA_FLAG_TE3;
  // Red LED off
  Control_RGB_LEDs(0, 0, 0);
}
```

Listing 9.11 Interrupt service routine for DMA2 channel 3 briefly lights the red LED.

Listing 9.11 shows the interrupt service routine that executes after the DMA controller completes its transfer of all data. Changing the red LED is optional. It is done so we can see when the ISR runs by using an oscilloscope or logic analyzer. The ISR checks the flag of the DMA to know if the interrupt has been triggered due to a completed transfer. Next, the code clears the DMA peripheral's flags to tell the peripheral the interrupt is being serviced. The last step is optional, turning off the red LED to indicate the ISR has completed.

Analysis

Let's verify the code and peripherals generate the waveform correctly. Figure 9.8 shows the analog waveform output (upper trace) and the DMA ISR activity (lower trace). The waveform repeats every 512 samples `(NUM_STEPS)` at 10 µs per sample, or every 5.12 ms. The DMA ISR runs at the end of every 512 transfers.

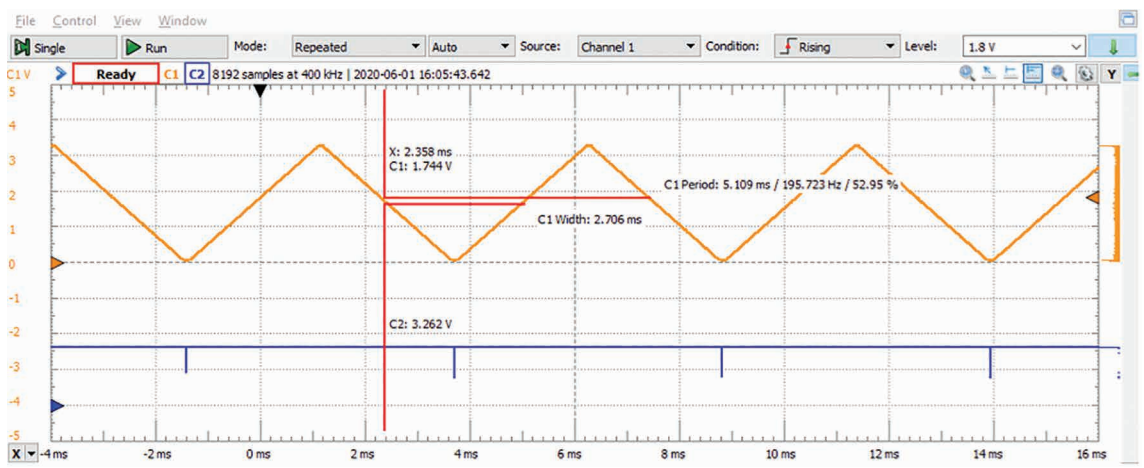

Figure 9.8 Analog output signal (upper trace) and DMA ISR activity (lower) show correct waveform generation. ISR is not strictly necessary for this example.

The DMA controller takes over the bus to transfer each sample, preventing the CPU from using the bus. According to the DMA application note, each transfer takes about five cycles every 10 µs, so the maximum fraction of time that DMA uses the bus matrix and can interfere with the CPU is 5 cycles/(48 MHz × 10 µs) = 1.04%.

If it is used, the ISR will also take up some time. In Figure 9.9 we zoom in and see the ISR is active for about 1.77 µs. There is a minor overhead to enter and exit the ISR, adding roughly 20 cycles, or about 0.4 µs at 48 MHz. This is about (1.77 µs + 0.4 µs) × 194 Hz = 0.00042 or 0.042% of processor time.

Adding these two values together shows that this waveform generator should slow the CPU by less than 1.1%, leaving over 98.9% available for other processing.

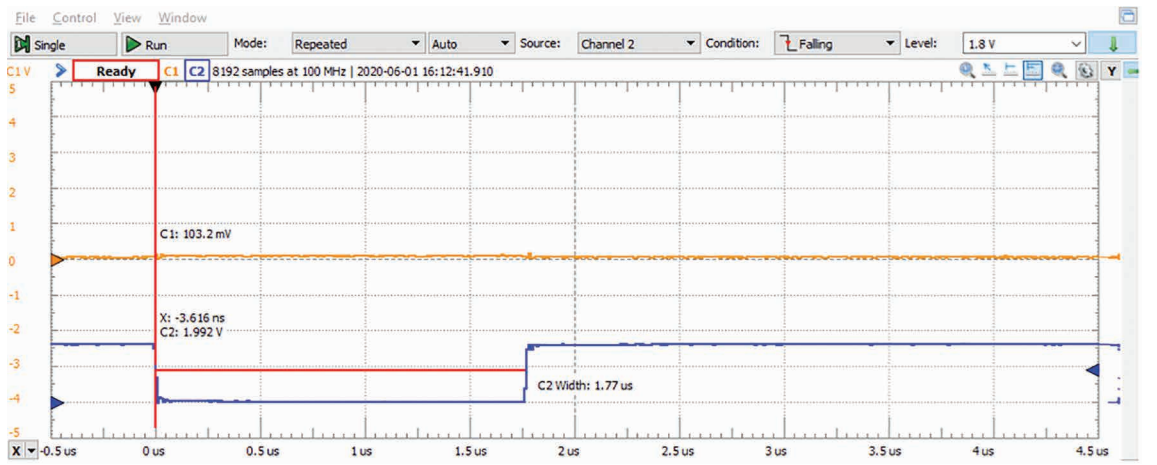

Figure 9.9 DMA ISR is active for about 1.77 µs.

Compare this with the approach from Chapter 7 based on the timer ISR. The total CPU load including interrupt overhead is (1.6 µs + 0.4 µs) × 100 kHz = 0.20 = 20%. Using the DMA system has reduced overhead by a factor of nearly twenty.

Summary

The DMA system can transfer information among peripherals and memory quickly. In many cases this can eliminate the need to use software on the CPU, improving system responsiveness, predictability, and throughput while freeing up time for the CPU either to use on other activities or to save power by sleeping.

Exercises

1. Determine and present the register configuration needed so that the next 1024 bytes of data received on USART1 are saved by the DMA1 controller in memory starting at address `DestAddress`. When the transfer is complete, the DMA controller must generate an interrupt.
2. Consider the memory transfer example. Rewrite the code to copy the contents of array `s` to array `d` without using pointers and measure its timing. How long does it take per array element, and how does this compare with the transfer using DMA? Enable maximum optimization for speed in the compiler, rebuild the code, and repeat the measurement. Are the results different, and if so, how do they compare with the transfer using DMA?
3. Determine and present the DMA1 control register configuration needed so that the next 15000 ADC results are sent out through SPI1 using DMA1 channel 2. Assume the ADC has been configured to perform 8-bit conversions and generate an interrupt upon completion.
4. We wish to increase the sampling rate for the analog waveform generator in this chapter. What is the maximum sampling rate possible that leaves 50% of the CPU's time for other processing?

References

[1] STMicroelectronics NV, *Application Note AN2548: Using the STM32F0/F1/F3/G0/Lx Series DMA Controller*, rev. 6, 2019.
[2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.